

Hard-Coding tools.py in Python

EOQ, safety stock, SKU lookup, and doc search -- each wrapped as a LangChain @tool.

Project: logistics-rag-agent

File: tools.py

Role: Give the agent real actions instead of guessing numbers

Tools are Python functions the LLM can call. Docstrings matter: the model reads them to decide WHEN to call each tool and WHICH arguments to pass.

1. Big picture

- eoq_calculator -- classic inventory math
- safety_stock_estimator -- buffer stock under uncertainty
- lookup_sample_inventory -- CSV (or hardcoded fallback) SKU table
- search_logistics_docs -- RAG retrieval packaged as a tool
- get_tools() -- list handed to agent.py

WHY

Calculators belong in tools, not in free-form LLM arithmetic. The model chooses the tool; Python does the math correctly.

2. Build step by step

Step 1: Imports and inventory path

```
from __future__ import annotations

import csv
import logging
import math
from pathlib import Path

from langchain.tools import tool

logger = logging.getLogger("logistics_rag")

INVENTORY_CSV = Path(__file__).parent / "data" / "sample_inventory.csv"
```

- @tool decorator -- turns a function into a LangChain Tool (name, schema, description)
- csv + Path -- read sample inventory without a database

WHY

The function docstring becomes the tool description the LLM sees. Write 'Use when...' guidance in the docstring on purpose.

Step 2: Hardcoded fallback inventory

```
_SAMPLE_INVENTORY: dict[str, dict] = {
    "SKU-1001": {"description": "...", "on_hand": 420, ...},
    "SKU-1002": {...},
}
```

Leading underscore means 'internal'. If the CSV is missing, demos still work.

WHY

Demo resilience: workshops and CI should not fail just because someone forgot the CSV.

Step 3: Load CSV into a dict keyed by SKU

```
def _load_inventory() -> dict[str, dict]:
    if not INVENTORY_CSV.exists():
        return {k: dict(v) for k, v in _SAMPLE_INVENTORY.items()}

    rows: dict[str, dict] = {}
    with INVENTORY_CSV.open(newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            sku = row["sku"].strip().upper()
            rows[sku] = {
                "description": row["description"],
```

```
        "on_hand": int(row["on_hand"]),
        "reorder_point": int(row["reorder_point"]),
        "lead_time_days": int(row["lead_time_days"]),
        "unit_cost": float(row["unit_cost"]),
    }
    return rows
```

- newline="" -- required csv module contract on all platforms
- .strip().upper() -- normalize SKU keys so 'sku-1001' still matches
- int()/float() -- CSV cells are strings until you cast them
- dict(v) copy on fallback -- avoid accidental mutation of the module constant

WHY

Casting at load time fails fast on bad CSV data instead of blowing up later inside a tool call.

Step 4: EOQ tool

```
@tool
def eq_calculator(annual_demand: float, order_cost: float, holding_cost: float) -> dict:
    """Calculate classic Economic Order Quantity (EOQ).
    ...
    Formula:  $EOQ = \sqrt{2 * D * S / H}$ 
    """
    if annual_demand <= 0 or order_cost <= 0 or holding_cost <= 0:
        return {"error": "annual_demand, order_cost, and holding_cost must all be > 0"}

    eq = math.sqrt(2 * annual_demand * order_cost / holding_cost)
    orders_per_year = annual_demand / eq
    return {
        "eq": round(eq, 2),
        "orders_per_year": round(orders_per_year, 2),
        ...
        "formula": "sqrt(2 * D * S / H)",
    }
```

- @tool reads type hints to build an args schema for the LLM
- Validate > 0 before sqrt -- avoids ZeroDivision / domain errors
- Return dict (not just a float) -- richer context for the agent's final wording
- Include formula string -- transparency for the user

WHY

Returning {'error': ...} instead of raising lets the agent narrate the problem instead of crashing the whole turn.

Step 5: Safety stock tool

```
@tool
def safety_stock_estimator(
    demand_std: float,
    lead_time: float,
    z_score: float = 1.65,
) -> dict:
    ...
    safety_stock = z_score * demand_std * math.sqrt(lead_time)
```

- Default z_score=1.65 -- common approx for ~95% cycle service level
- demand_std and lead_time must use the SAME time unit (days with daily std, etc.)

WHY

Defaults in the signature teach the model a sensible value when the user omits z.

Step 6: SKU lookup tool

```
@tool
def lookup_sample_inventory(sku: str) -> dict:
    inventory = _load_inventory()
    key = sku.strip().upper()
    if key not in inventory:
        return {"error": ..., "known_skus": known}
    record = {"sku": key, **inventory[key]}
    record["below_reorder_point"] = record["on_hand"] < record["reorder_point"]
    record["status"] = "REORDER" if below_rop else "OK"
    return record
```

- On miss, return known_skus -- agent can tell the user what exists

- Derived fields (below_reorder_point, status) -- business logic in Python, not LLM

WHY

Never ask the LLM to 'remember' inventory. Look it up. That is the whole point of tools.

Step 7: Doc search as a tool (lazy import)

```
@tool
def search_logistics_docs(query: str) -> str:
    """Search ingested logistics SOPs..."""
    from rag import TOP_K, format_docs, get_vectorstore, log_retrieved_chunks

    retriever = get_vectorstore().as_retriever(search_kwargs={"k": TOP_K})
    docs = log_retrieved_chunks(retriever.invoke(query))
    return format_docs(docs)
```

- Import inside the function -- avoids circular imports (rag <-> tools) at module load
- Returns formatted string -- same shape the grounded RAG path uses
- Docstring tells the agent NOT to use this for EOQ/SKU math

WHY

Routing policy lives in tool descriptions + agent system prompt. Clear boundaries reduce wrong-tool calls.

Step 8: Export the tool list

```
def get_tools() -> list:
    """All tools available to the logistics agent."""
    return [
        eoq_calculator,
        safety_stock_estimator,
        lookup_sample_inventory,
        search_logistics_docs,
    ]
```

WHY

One factory function means agent.py does not hardcode imports of each tool. Add a tool here and the agent picks it up.

3. Practice

- Call `eoq_calculator.invoke({'annual_demand': 1000, 'order_cost': 50, 'holding_cost': 2})`
- Lookup SKU-1002 -- confirm status REORDER
- Ask the agent an EOQ question and verify -v shows a tool call, not pure guessing

4. Common failures

- LLM ignores tool -- weak docstring / system prompt; tighten 'Use when...' text
- CSV parse error -- check header names match row['sku'] etc.
- Circular import -- keep rag import inside `search_logistics_docs`